

Functions and arguments

Project: Python readiness portfolio

1. Your mission

Your learning target

I can distinguish a function definition from a function call.

I can match positional arguments to parameters in order.

I can reuse one function and test that it uses the supplied values.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Lists and indexing. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Branching and boundary tests. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Distinguish a function definition from a function call

Match positional arguments to parameters in order

Reuse one function and test that it uses the supplied values

Retrieve what you already know

A script sets `capacity = 12` and `used = 4`. What does `print(capacity - used)` display?

My starting explanation

This week's end goal

Trace functions and arguments and explain the changed program before running it.

2. Notice the evidence

A fictional supply cupboard records capacity and the number of occupied spaces. The remaining-space calculation is capacity minus used. Instead of copying the same instructions for every cupboard, define one function and call it with different values. A function makes a procedure reusable; its name alone does not guarantee that its instructions are correct.

Our input contract requires nonnegative integer counts with used no greater than capacity. The example assumes these conditions; it does not validate them. This week the function prints its result. Week 9 will show how to return a value for later calculations.

List the given facts and one fact that is not supplied.

3. See how it works

```
def show_remaining(capacity, used):  
    remaining = capacity - used  
    print(remaining)  
  
show_remaining(12, 4)  
show_remaining(10, 7)
```

Arguments matched in order

Call · Parameter values · Printed output

Call: `show_remaining(12, 4)`

Parameter values: capacity 12; used 4

Printed output: 8

Call: `show_remaining(10, 7)`

Parameter values: capacity 10; used 7

Printed output: 3

Each call runs the same body with the values supplied for that call.

`def` introduces the definition. The function name is followed by parameter names in parentheses and a colon. Four-space indentation marks the body. Executing the definition creates the function but does not run its body. The two unindented calls then run it, producing 8 and 3 on separate lines. A blank line helps readability; indentation determines the block.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

Using the correct definition, predict `show_remaining(8, 3)`, `show_remaining(8, 8)` and `show_remaining(8, 0)` in that order. Explain what happens if the file contains only the definition.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: Remove both calls and leave the definition only.

Prediction: No output is printed.

Reason: The body runs only when the function is called.

5. Words that unlock the idea

Word · Meaning

Word: Function

Meaning: A named block of reusable instructions that runs when called

Word: Definition

Meaning: Code that creates a function and specifies its body

Word: Call

Meaning: An instruction that runs a function with supplied arguments

Word: Parameter

Meaning: A name in a function definition that receives an argument

Word: Argument

Meaning: A value supplied when calling a function

Use two terms to explain a step in this week's example.

6. Follow a worked example

Execution event · What happens

Execution event: Execute the definition

What happens: Create the function; no output yet

Execution event: Call with 12 and 4

What happens: Match parameters; subtract; print 8

Execution event: Call with 10 and 7

What happens: Match new values; subtract; print 3

Execution event: After the second call

What happens: Continue with the next statement

In `show_remaining(12, 4)`, 12 and 4 are arguments; `capacity` and `used` are parameters. Positional arguments match from left to right. `show_remaining(4, 12)` produces -8, showing why order matters; it also violates our written input contract. Use a fresh file with the definition before its calls.

Predict the output of `show_remaining(9, 9)` followed by `show_remaining(9, 0)`. Name the parameter receiving each argument. What would be printed if both calls were removed?

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Functions and arguments

Match arguments to parameters. A function prints the difference between `capacity` and `reserved` spaces.

The facts we will use

Python 3: `def show_free(capacity,reserved): print(capacity-reserved)`. Calls: `show_free(11,3); show_free(3,11)`.

Functions and arguments

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Define without calling

The definition creates the function but prints nothing yet.



2. Match first arguments

capacity=11 and reserved=3 give a printed 8.



3. Trace reversed call

capacity=3 and reserved=11 give -8 under the same arithmetic.

CHANGE ONE THING

Remove both calls and leave the definition only.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Define without calling

What happens and why: The definition creates the function but prints nothing yet.

Step: 2. Match first arguments

What happens and why: capacity=11 and reserved=3 give a printed 8.

Step: 3. Trace reversed call

What happens and why: capacity=3 and reserved=11 give -8 under the same arithmetic.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Positional arguments match parameter names in order. A function definition is not an execution of its body; separate calls supply separate input values.

What this example does not establish

The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

7. Find and repair a mistake

```
def show_remaining(capacity, used):  
    remaining = 12 - 4  
    print(remaining)  
  
show_remaining(12, 4)  
show_remaining(10, 7)
```

This faulty function prints 8 twice. Its fixed numbers ignore the arguments. Replace 12 - 4 with capacity - used, then rerun both calls. Testing only the first call would miss the bug because its expected answer happens to be 8.

For this two-parameter definition, show_remaining(12) raises TypeError because an argument is missing. Correct the call to supply both values. Naming a function without calling it does not execute the body: the parentheses in show_remaining(12, 4) are part of the call.

Locate the earliest unsupported step

The faulty body calculates 12 - 4 regardless of the call. State the output and expected output for show_remaining(11, 2). Repair the expression and name an earlier call to rerun.

State one repair and the checks you would rerun.

8. Practise together

In `def show_remaining(capacity, used)`, identify the function name and parameters. In `show_remaining(15, 6)`, identify the arguments and their matching parameters.

Predict the output of `show_remaining(9, 9)` followed by `show_remaining(9, 0)`. Name the parameter receiving each argument. What would be printed if both calls were removed?

Explain your prediction to a partner before comparing results.

Practise with less help: Follow the execution order

Use workbook W2 for the connected example “Functions and arguments”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case.

Each person writes their own explanation.

Check your reasoning

Use the worked case for Functions and arguments. Proposed explanation: “Writing def runs the body immediately”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Match every argument to a parameter for the two calls. Why do outputs differ, and what prints when both calls are removed?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

Using the correct definition, predict `show_remaining(8, 3)`, `show_remaining(8, 8)` and `show_remaining(8, 0)` in that order. Explain what happens if the file contains only the definition.

```
def show_remaining(capacity, used):  
    remaining = capacity - used  
    print(remaining)  
  
show_remaining(12, 4)  
show_remaining(10, 7)
```

```
def show_remaining(capacity, used):  
    remaining = 12 - 4  
    print(remaining)  
  
show_remaining(12, 4)
```

```
show_remaining(10, 7)
```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

In `def show_remaining(capacity, used)`, identify the function name and parameters. In `show_remaining(15, 6)`, identify the arguments and their matching parameters.

The faulty body calculates $12 - 4$ regardless of the call. State the output and expected output for `show_remaining(11, 2)`. Repair the expression and name an earlier call to rerun.

The correct subtraction function prints `-1` for `show_remaining(4, 5)`. Does successful execution make this a

valid cupboard record? Explain what the function assumes and what needs checking.

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled functions and arguments evidence sheet to Python readiness portfolio. Show how you can distinguish a function definition from a function call. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record: _____

Project evidence: Inputs permitted at prediction time

Record: _____

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

12. Show what you know

For each call, record the supplied arguments, expected printed output and observed output. Include different values that produce different answers, plus a valid zero-result case. Call order controls output order; it does not change the subtraction rule. These functions use supplied data and written instructions, not a pattern learned from examples.

A learner tests only `show_remaining(12, 4)` and says the function works for every cupboard. Give a second valid call that exposes the fixed-number bug, state its expected output and explain what passing two tests still cannot prove.

The correct subtraction function prints -1 for `show_remaining(4, 5)`. Does successful execution make this a valid cupboard record? Explain what the function assumes and what needs checking.

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

Arguments matched in order

Call · Parameter values · Printed output

Call: `show_remaining(12, 4)`

Parameter values: capacity 12; used 4

Printed output: 8

Call: show_remaining(10, 7)

Parameter values: capacity 10; used 7

Printed output: 3

Each call runs the same body with the values supplied for that call.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Match positional arguments to parameters

Predict both returned and printed values, showing which argument supplies first and second. What is displayed if the program contains only the function definition and no calls? Why does swapping these arguments change the result?

```
def difference(first, second):  
    return first - second  
  
print(difference(11, 4))  
  
print(difference(4, 11))
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Each sealed box contains 6 counters. Define `show_stock(boxes, loose)` to print the total counters using its two parameters. Assume nonnegative integer arguments. Call it with (3, 2), (0, 2) and (3, 0). Predict each output and explain why `show_stock(2, 3)` is a different case.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.